

《数据库系统原理》实验报告

郑州大学计算机与人工智能学院

(2025-2026 年第二学期)

专业班级： 人工智能 2 班

学 号： 202315060230

姓 名： 齐迎斌

指导教师： 胡世哲

最终成绩：

实验 1 认识 DBMS 系统

1. 实验目的

- 1) 通过对某个商用数据库管理系统（OpenGauss 等）的使用，了解 DBMS 的工作原理和系统构架。
- 2) 熟悉对 DBMS 的操作。
- 3) 搭建今后实验的平台。

2. 实验内容

1) 安装和启动

了解 OpenGauss 的三种安装模式：华为弹性云上安装、虚拟机一键安装和虚拟机安装 openEuler 后再安装 OpenGauss。了解了这三种安装模式之后，选择一个适合自己的安装方式，安装 OpenGauss，为下一步实验做准备。

2) 数据库系统的构架

了解 OpenGauss 数据库系统的启动和连接数据库，postgres 为 OpenGauss 安装完成后默认生成的数据库。初始可以连接到此数据库进行新数据库的创建。使用数据库前，需先使用客户端程序，gsql 是 OpenGauss 数据库提供的命令行方式的数据库连接工具。

3) 创建数据库用户和数据库

默认只有 OpenGauss 安装时创建的管理员用户可以访问初始数据库，还可以创建其他数据库用户帐号。使用新用户连接到此数据库执行接下来的创建表等操作，也可以选择继续在默认的 postgres 数据库下做后续的体验。

实验过程：

1. 虚拟软件 VirtualBox 的安装

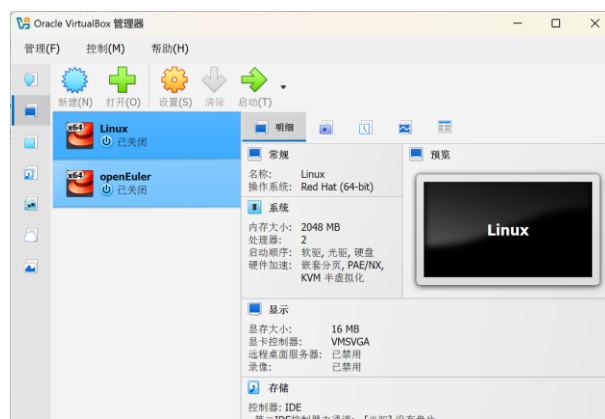


图 1-1 VirtualBox 虚拟机管理界面

2. openEuler 操作系统和 openGauss 数据库的安装（使用磁盘映像文件）

（1）下载磁盘映像文件

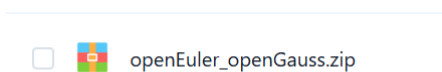


图 1-4 （1）下载磁盘映像文件

如图 1-4 所示，该步骤展示了“（1）下载磁盘映像文件”的执行结果。

（2）将映像文件装入 VirtualBox



图 1-2 openEuler 镜像文件加载界面

3. 连接数据库

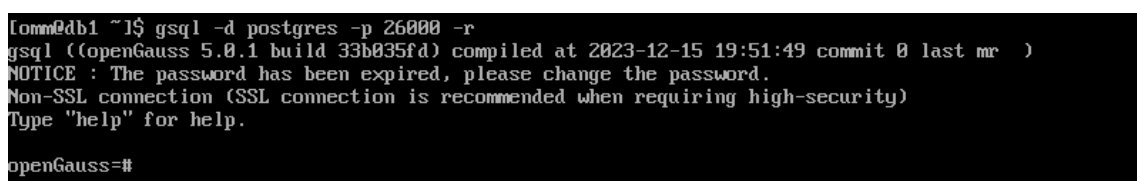


图 1-3 gsql 连接数据库成功界面

问题：虚拟机环境和数据库连接配置不够熟悉，导致系统运行较慢、镜像挂载和数据库连接过程中出现操作不顺利的情况。

解决办法：针对这些问题，我通过调整虚拟机内存和 CPU 配置、重新检查 ISO 镜像挂载情况、确认数据库服务是否正常启动，并核对用户名、数据库名和连接命令，最终成功完成了 openEuler 系统安装、OpenGauss 数据库启动以及 gsql 客户端连接。

3. 实验分析与总结

分析：通过本次实验，我了解了 OpenGauss 数据库管理系统的基本安装方式和运行流程，掌握了在虚拟机环境下安装 openEuler 操作系统和 OpenGauss 数据库的方法。通过启动数据库服务并使用 gsql 客户端连接数据库，我初步认识了数据库服务器、客户端工具、数据库用户和数据库之间的关系。

总结：本次实验完成了数据库实验平台的搭建，熟悉了数据库的启动、连接、创建用户和创建数据库等基本操作。通过实验，我认识到 DBMS 不仅负责数据存储和管理，还涉及用户权限、系统环境、客户端连接等多个方面，为后续数据库实验奠定了基础。

心得体会

本实验使我初步掌握了 DBMS 环境搭建、数据库启动和客户端连接流程，为后续实验奠定了基础。

实验 2 创建数据库、表和索引

1. 实验目的

- 1) 掌握如何在 OpenGauss 中创建用户、数据库、模式、表和索引等。
- 2) 灵活运用 SQL 语句建立上述概念。
- 3) 理解数据库用户、数据库、schema、表、索引之间的关系。

2. 实验内容

- 1) 创建数据库、模式、表和索引。

创建数据库 S_C:

```
openGauss=# CREATE DATABASE S_C;
CREATE DATABASE
openGauss=# \l
```

Name	Owner	Encoding	Collate	Ctype	Access privileges
postgres	omm	UTF8	C	C	
s_c	omm	UTF8	C	C	
template0	omm	UTF8	C	C	=c/omm + omm=CTc/omm
template1	omm	UTF8	C	C	=c/omm + omm=CTc/omm

(4 rows)

图 2-1 创建数据库 S_C 结果

创建表 course\departments\sc\students\teachers\teaches:

```
openGauss=> \dt
```

Schema	Name	Type	Owner	Storage
student	courses	table	student	{orientation=row,compression=no}
student	departments	table	student	{orientation=row,compression=no}
student	sc	table	student	{orientation=row,compression=no}
student	students	table	student	{orientation=row,compression=no}
student	teachers	table	student	{orientation=row,compression=no}
student	teaches	table	student	{orientation=row,compression=no}

(6 rows)

图 2-2 查看数据库中基本表结果

```
openGauss=> -- 创建Departments表
openGauss=> CREATE TABLE Departments(
openGauss(> Dno CHAR(4) PRIMARY KEY
openGauss(> Dname VARCHAR(20),
openGauss(> Dheadno CHAR(7)
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index on Departments
CREATE TABLE
```

```
openGauss=> -- 创建Teachers表
openGauss=> CREATE TABLE Teachers(
openGauss(> Tno CHAR(7) PRIMARY KEY,
openGauss(> Tname CHAR(10) NOT NULL,
openGauss(> Sex CHAR(3) CHECK(Sex = '男' OR Sex = '女'),
openGauss(> Birthday DATE,
openGauss(> Title VARCHAR(15),
openGauss(> Dno CHAR(4),
openGauss(> FOREIGN KEY (Dno) REFERENCES Departments(Dno)
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index on Teachers
CREATE TABLE
```

图 2-3 创建 Departments 表结果

图 2-4 创建 Teachers 表结果

```
openGauss=> -- 创建Courses表
openGauss=> CREATE TABLE Courses(
openGauss(> Cno CHAR(5) PRIMARY KEY,
openGauss(> Cname VARCHAR(40) NOT NULL
openGauss(> Period SMALLINT,
openGauss(> Credit SMALLINT
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index
CREATE TABLE
```

```
openGauss=> -- 创建Students表
openGauss=> CREATE TABLE Students(
openGauss(> Sno CHAR(9) PRIMARY KEY,
openGauss(> Sname VARCHAR(10) NOT NULL,
openGauss(> Sex CHAR(3) CHECK(Sex = '男' OR Sex = '女'),
openGauss(> Birthday DATE,
openGauss(> Speciality VARCHAR(20),
openGauss(> Dno CHAR(4),
openGauss(> FOREIGN KEY (Dno) REFERENCES Departments(Dno)
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index
CREATE TABLE
```

图 2-5 创建 Courses 表结果

图 2-6 创建 Students 表结果

```
openGauss=> -- 创建Teaches表
openGauss=> CREATE TABLE Teaches(
openGauss(> Tno CHAR(7),
openGauss(> Cno CHAR(5),
openGauss(> TCscore SMALLINT,
openGauss(> PRIMARY KEY (Tno, Cno),
openGauss(> FOREIGN KEY (Tno) REFERENCES Teachers(Tno),
openGauss(> FOREIGN KEY (Cno) REFERENCES Courses(Cno)
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index
CREATE TABLE
```

```
openGauss=> -- 创建SC表
openGauss=> CREATE TABLE SC(
openGauss(> Sno CHAR(9),
openGauss(> Cno CHAR(5),
openGauss(> Grade SMALLINT CHECK(Grade >= 60),
openGauss(> PRIMARY KEY (Sno, Cno),
openGauss(> FOREIGN KEY (Sno) REFERENCES Students(Sno),
openGauss(> FOREIGN KEY (Cno) REFERENCES Courses(Cno)
openGauss(> );
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index
CREATE TABLE
```

图 2-7 创建 Teaches 表结果

图 2-8 创建 SC 表结果

2) 修改基本表的结构。

在 OpenGauss 数据库系统中使用 SQL 语句（按教材上所示的属性）建立基本表，如 Students 表、Courses 表、Teachers 表、Departments 表、SC 表等，并向表中填充数据。熟悉使用虚拟机进行 OpenGauss 中 SQL 语句的代码输入方式。

填充和查看表：

```
openGauss=> -- 填充Departments表
openGauss=> INSERT INTO Departments (Dno, Dname)
openGauss-> VALUES ('CS', '计算机系'),
openGauss-> ('MA', '数学系'),
openGauss-> ('PH', '物理系'),
openGauss-> ('CH', '化学系');
INSERT 0 4
openGauss=> -- 填充Teachers表
openGauss=> INSERT INTO Teachers (Tno, Tname, Sex, Birthday, Title, Dno)
openGauss-> VALUES ('05001', '张海峰', '男', '1969-02-21', '教授', 'CS'),
openGauss-> ('05002', '刘晓明', '男', '1978-10-06', '副教授', 'CS'),
openGauss-> ('05003', '王梅', '女', '1972-02-25', '副教授', 'CS'),
openGauss-> ('05004', '赵青山', '男', '1981-05-18', '讲师', 'CS'),
openGauss-> ('06001', '朱杰', '男', '1963-01-30', '教授', 'MA'),
openGauss-> ('06002', '欧阳梅', '女', '1970-08-11', '副教授', 'MA');
INSERT 0 6
openGauss=> -- 修改Departments表
openGauss=> UPDATE Departments SET Dheadno = '05001' WHERE Dno = 'CS';
UPDATE 1
openGauss=> UPDATE Departments SET Dheadno = '06001' WHERE Dno = 'MA';
UPDATE 1
openGauss=> -- 填充Students表
openGauss=> INSERT INTO Students (Sno, Sname, Sex, Birthday, Speciality, Dno)
openGauss-> VALUES ('202405001', '张华', '男', '2006-12-16', '计算机', 'CS'),
openGauss-> ('202405002', '李阳', '女', '2005-10-17', '计算机', 'CS'),
openGauss-> ('202405003', '欧阳山', '男', '2006-05-14', '计算机', 'CS'),
openGauss-> ('202406001', '林艳', '女', '2005-09-20', '数学', 'MA'),
openGauss-> ('202406002', '高山', '男', '2004-08-16', '数学', 'MA'),
openGauss-> ('202406003', '王海洋', '男', '2004-01-30', '数学', 'MA');
INSERT 0 6
```

```
openGauss=> SELECT * FROM Students;
sno | sname | sex | birthday | speciality | dno
-----+-----+----+-----+-----+----
202405001 | 张华 | 男 | 2006-12-16 00:00:00 | 计算机 | CS
202405002 | 李阳 | 女 | 2005-10-17 00:00:00 | 计算机 | CS
202405003 | 欧阳山 | 男 | 2006-05-14 00:00:00 | 计算机 | CS
202406001 | 林艳 | 女 | 2005-09-20 00:00:00 | 数学 | MA
202406002 | 高山 | 男 | 2004-08-16 00:00:00 | 数学 | MA
202406003 | 王海洋 | 男 | 2004-01-30 00:00:00 | 数学 | MA
(6 rows)

openGauss=> SELECT * FROM Courses;
cno | cname | period | credit
-----+-----+-----+-----
CS101 | 计算机科学导论 | 2 | 1
CS102 | C语言 | 4 | 3
CS201 | 数据结构 | 4 | 3
CS202 | 离散数学 | 4 | 3
CS301 | 数据库系统原理 | 4 | 3
(5 rows)

openGauss=> SELECT * FROM SC;
sno | cno | grade
-----+-----+-----
202405001 | CS101 | 89
202405002 | CS101 | 91
202405003 | CS101 | 80
202405001 | CS102 | 90
202405002 | CS102 | 87
202405003 | CS102 | 82
202405001 | CS201 | 88
202405002 | CS201 | 89
202405003 | CS201 | 90
202405001 | CS202 | 86
202405002 | CS202 | 85
202405001 | CS301 | 92
(12 rows)
```

图 2-9 插入并查询表中数据结果

3) 创建和删除索引

使用 SQL 语句在上述表中创建索引，熟悉相应的 SQL 语句，请参考教材 P73-79 页的 SQL 语句，请注意标准 SQL 和 OpenGauss 中相应 SQL 语句相同与不同之处。

创建索引：

```
(4 rows)
openGauss=> CREATE INDEX idx_Student_Sname ON Students(Sname);
CREATE INDEX
openGauss=> \di
```

List of relations					
Schema	Name	Type	Owner	Table	Storage
student	courses_pkey	index	student	courses	
student	departments_pkey	index	student	departments	
student	idx_student_sname	index	student	students	
student	sc_pkey	index	student	sc	
student	students_pkey	index	student	students	
student	teachers_pkey	index	student	teachers	
student	teaches_pkey	index	student	teaches	

```
(7 rows)
```

图 2-10 创建索引结果

删除索引：

```
openGauss=> DROP INDEX idx_student_sname
openGauss-> ;
DROP INDEX
```

图 2-11 删除索引结果

3. 实验分析与总结

在实验过程中，首先通过启动数据库集群并连接到默认数据库，逐步熟悉了 gsql 命令行工具的基本使用方法。随后按照实验要求创建了 Departments、Teachers、Students、Courses、SC 和 Teaches 等多个基本表，并在建表过程中设置了主键、外键以及检查约束，从而建立起较为完整的关系模型。在具体操作中，可以明显体会到关系数据库中各表之间的联系是通过外键来实现的，例如 Students 表和 Teachers 表都通过 Dno 与 Departments 表建立关联，而 SC 表和 Teaches 表则通过复合主键与其他表形成多对多关系。

通过本次实验，掌握了在 OpenGauss 中创建数据库、基本表及索引的基本方法，理解了主键、外键及索引之间的关系，并熟悉了常用 SQL 语句的使用。实验过程中虽然遇到了一些操作错误，但通过调试与修改，提高了实际动手能力。

问题及解决办法

问题：建表时容易出现字段类型、主键和外键设置不一致的问题。

解决办法：对照表结构逐项检查字段名称、数据类型和约束条件，修改 SQL 语句后重新执行。

心得体会：本实验加深了我对数据库、表、索引以及表间关系的理解，提高了 SQL 建模能力。

实验 3 交互式 SQL 语句

1. 实验目的

- 1) 熟悉数据库的交互式 SQL 工具。
- 2) 熟悉通过 SQL 语句操作数据库。
- 3) 完成作业的上机练习。

2. 实验内容

1) 查询操作

根据教材 P79-94 页所有查询实例，完成各类查询操作。

-- 例 4.8 查询所有课程的信息

```
Database changed
mysql> SELECT *
-> FROM Courses;
+-----+-----+-----+-----+
| Cno | Cname | Period | Credit |
+-----+-----+-----+-----+
| CS101 | 计算机科学导论 | 2 | 1 |
| CS102 | C语言 | 4 | 3 |
| CS201 | 数据结构 | 4 | 3 |
| CS202 | 离散数学 | 4 | 3 |
| CS301 | 数据库系统原理 | 4 | 3 |
+-----+-----+-----+-----+
5 rows in set (0.009 sec)
```

图 3-1 查询所有课程信息结果

-- 例 4.9 查询每位学生的年龄

```
mysql> SELECT Sname, 2017 - YEAR(Birthday) AS Age
-> FROM Students;
+-----+-----+
| Sname | Age |
+-----+-----+
| 张华 | 18 |
| 李玉 | 19 |
| 欧阳山 | 18 |
| 林艳 | 19 |
| 高山 | 20 |
| 王海洋 | 20 |
+-----+-----+
6 rows in set (0.019 sec)
```

图 3-2 查询学生年龄结果

-- 例 4.10 (1) 查询职称为讲师的全体教师姓名和性别

```
mysql> SELECT Tname, Sex
-> FROM Teachers
-> WHERE Title = '讲师';
+-----+-----+
| Tname | Sex |
+-----+-----+
| 赵青山 | 男 |
+-----+-----+
1 row in set (0.022 sec)
```

图 3-3 查询讲师信息结果

-- 例 4.11 (1) 查询出生年份在 1997~1999 年的学生姓名和专业

```
mysql> SELECT Sname, Speciality
-> FROM Students
-> WHERE YEAR(Birthday) BETWEEN 1997 AND 1999;
```

Sname	Speciality
张华	计算机
李玉	计算机
欧阳山	计算机
林艳	数学
高山	数学
王海洋	数学

6 rows in set (0.014 sec)

图 3-4 查询指定出生年份学生结果

-- 例 4.12 (1) 查询计算机科学与技术 and 软件工程专业学生的学号和姓名

```
mysql> SELECT Sno, Sname
-> FROM Students
-> WHERE Speciality NOT IN ('计算机科学与技术', '软件工程');
```

Sno	Sname
201705001	张华
201705002	李玉
201705003	欧阳山
201706001	林艳
201706002	高山
201706003	王海洋

6 rows in set (0.006 sec)

图 3-5 查询指定专业学生结果

-- 例 4.13 (1) 查询所有以“数据”开头的课程名

```
mysql> SELECT Cname
-> FROM Courses
-> WHERE Cname LIKE '数据%';
```

Cname
数据结构
数据库系统原理

2 rows in set (0.046 sec)

图 3-6 模糊查询课程名结果

-- 例 4.14 查询成绩为空的学生学号和课程号

```
mysql> SELECT Sno, Cno
-> FROM SC
-> WHERE Grade IS NULL;
Empty set (0.007 sec)

mysql>
```

图 3-7 查询空成绩结果

-- 例 4.15 (1) 查询每位学生 CS202 课程的成绩，并按成绩降序排序

```
mysql> SELECT *
-> FROM SC
-> WHERE Cno = 'CS202'
-> ORDER BY Grade DESC;
+-----+-----+-----+
| Sno      | Cno   | Grade |
+-----+-----+-----+
| 201705001 | CS202 | 86    |
| 201705002 | CS202 | 85    |
+-----+-----+-----+
2 rows in set (0.010 sec)
```

图 3-8 按成绩降序查询结果

-- 例 4.16 (1) 查询选修 CS302 课程的学生人数

```
mysql> SELECT COUNT(*)
-> FROM SC
-> WHERE Cno = 'CS302';
+-----+
| COUNT(*) |
+-----+
| 0        |
+-----+
1 row in set (0.061 sec)
```

图 3-9 统计选课人数结果

-- 例 4.17 查询每个学生的平均成绩

```
mysql> SELECT Sno, AVG(Grade)
-> FROM SC
-> GROUP BY Sno;
+-----+-----+
| Sno      | AVG(Grade) |
+-----+-----+
| 201705001 | 89.0000    |
| 201705002 | 88.0000    |
| 201705003 | 84.0000    |
+-----+-----+
3 rows in set (0.020 sec)
```

图 3-10 分组查询平均成绩结果

-- 例 4.18 错误写法：WHERE 中不能使用聚集函数

正确写法：使用 HAVING

```
mysql> SELECT Sno, AVG(Grade)
-> FROM SC
-> WHERE AVG(Grade) > 85
-> GROUP BY Sno;
ERROR 1111 (HY000): Invalid use of group function
mysql> SELECT Sno, AVG(Grade)
-> FROM SC
-> GROUP BY Sno
-> HAVING AVG(Grade) > 85;
+-----+-----+
| Sno      | AVG(Grade) |
+-----+-----+
| 201705001 | 89.0000    |
| 201705002 | 88.0000    |
+-----+-----+
2 rows in set (0.020 sec)
```

图 3-11 使用 HAVING 查询结果

-- 例 4.19 查询学号为 201705001 的学生各科成绩，显示课程名和成绩

```
mysql> SELECT Cname, Grade
-> FROM SC, Courses
-> WHERE SC.Cno = Courses.Cno
-> AND Sno = '201705001';
+-----+-----+
| Cname      | Grade |
+-----+-----+
| 计算机科学导论 | 89    |
| C语言      | 90    |
| 数据结构    | 88    |
| 离散数学    | 86    |
| 数据库系统原理 | 92    |
+-----+-----+
5 rows in set (0.007 sec)
```

图 3-12 多表连接查询成绩结果

-- 例 4.20 查询选修 CS202 且成绩在 90 分以上的学生学号、姓名和成绩

```
mysql> SELECT Students.Sno, Sname, Grade
-> FROM Students, SC
-> WHERE Students.Sno = SC.Sno
-> AND Cno = 'CS202'
-> AND Grade > 90;
Empty set (0.013 sec)
```

图 3-13 嵌套查询结果

-- 例 4.21 查询每个学生选修的每门课程成绩，列出学号、姓名、课程名和成绩

```
mysql> SELECT Students.Sno, Sname, Cname, Grade
-> FROM Students, SC, Courses
-> WHERE Students.Sno = SC.Sno
-> AND SC.Cno = Courses.Cno;
```

Sno	Sname	Cname	Grade
201705001	张华	计算机科学导论	89
201705002	李玉	计算机科学导论	91
201705003	欧阳山	计算机科学导论	80
201705001	张华	C语言	90
201705002	李玉	C语言	87
201705003	欧阳山	C语言	82
201705001	张华	数据结构	88
201705002	李玉	数据结构	89
201705003	欧阳山	数据结构	90
201705001	张华	离散数学	86
201705002	李玉	离散数学	85
201705001	张华	数据库系统原理	92

12 rows in set (0.006 sec)

图 3-14 集合查询结果

-- 例 4.22 查询每个学生平均成绩，输出平均成绩大于 85 分的学生学号、姓名和平均成绩

```
mysql> SELECT Students.Sno, Sname, AVG(Grade)
-> FROM SC, Students
-> WHERE Students.Sno = SC.Sno
-> GROUP BY Students.Sno, Sname
-> HAVING AVG(Grade) > 85;
```

Sno	Sname	AVG(Grade)
201705001	张华	89.0000
201705002	李玉	88.0000

2 rows in set (0.068 sec)

图 3-15 例 4.22 查询每个学生平均成绩，输出平均成绩大于 85 分结果

如图 3-15 所示，该步骤展示了“例 4.22 查询每个学生平均成绩，输出平均成绩大于 85 分结果”的执行结果。

-- 例 4.23 查询和林艳出生年份相同的学生姓名

```
mysql> SELECT S2.Sname
-> FROM Students S1, Students S2
-> WHERE S1.Birthday = S2.Birthday
-> AND S1.Sname = '林艳'
-> AND S2.Sname <> '林艳';
```

Empty set (0.008 sec)

图 3-16 例 4.23 查询和林艳出生年份相同的学生姓名

如图 3-16 所示，该步骤展示了“例 4.23 查询和林艳出生年份相同的学生姓名”的执行结果。

-- 例 4.24 查询和林艳在同一个专业学习的女同学学号和姓名

```
mysql> SELECT Sno, Sname
-> FROM Students
-> WHERE Sex = '女'
-> AND Speciality IN (
->     SELECT Speciality
->     FROM Students
->     WHERE Sname = '林艳'
-> );
```

Sno	Sname
201706001	林艳

1 row in set (0.056 sec)

图 3-17 例 4.24 查询和林艳在同一个专业学习的女同学学号和姓名

如图 3-17 所示，该步骤展示了“例 4.24 查询和林艳在同一个专业学习的女同学学号和姓名”的执行结果。

-- 例 4.25 查询比软件工程专业所有学生都小的其他专业学生学号、姓名、专业和出生日期

```
mysql> SELECT Sno, Sname, Speciality, YEAR(Birthday)
-> FROM Students
-> WHERE Speciality <> '软件工程'
-> AND YEAR(Birthday) > ALL (
->     SELECT YEAR(Birthday)
->     FROM Students
->     WHERE Speciality = '软件工程'
-> );
```

Sno	Sname	Speciality	YEAR(Birthday)
201705001	张华	计算机	1999
201705002	李玉	计算机	1998
201705003	欧阳山	计算机	1999
201706001	林艳	数学	1998
201706002	高山	数学	1997
201706003	王海洋	数学	1997

6 rows in set (0.065 sec)

图 3-18 例 4.25 查询比软件工程专业所有学生都小的其他专业学生结果

如图 3-18 所示，该步骤展示了“例 4.25 查询比软件工程专业所有学生都小的其他专业学生结果”的执行结果。

-- 例 4.26 查询平均成绩最高课程的课程号和平均成绩

```
mysql> -- 使用 ALL 查询平均成绩最高的课程
Query OK, 0 rows affected (0.004 sec)

mysql> SELECT Cno, AVG(Grade)
-> FROM SC
-> GROUP BY Cno
-> HAVING AVG(Grade) >= ALL (
->     SELECT AVG(Grade)
->     FROM SC
->     GROUP BY Cno
-> );
+-----+-----+
| Cno | AVG(Grade) |
+-----+-----+
| CS301 | 92.0000 |
+-----+-----+
1 row in set (0.010 sec)
```

图 3-19 例 4.26 查询平均成绩最高课程的课程号和平均成绩

如图 3-19 所示，该步骤展示了“例 4.26 查询平均成绩最高课程的课程号和平均成绩”的执行结果。

-- 例 4.27 查询所有选修了 CS403 课程的学生学号和姓名

```
mysql> SELECT Sno, Sname
-> FROM Students S
-> WHERE EXISTS (
->     SELECT *
->     FROM SC
->     WHERE Sno = S.Sno
->     AND Cno = 'CS403'
-> );
Empty set (0.006 sec)
```

图 3-20 例 4.27 查询所有选修了 CS403 课程的学生学号和结果

如图 3-20 所示，该步骤展示了“例 4.27 查询所有选修了 CS403 课程的学生学号和结果”的执行结果。

-- 例 4.28 查询选修了全部课程的学生学号和姓名

```
mysql> SELECT Sno, Sname
-> FROM Students S
-> WHERE NOT EXISTS (
->     SELECT *
->     FROM Courses C
->     WHERE NOT EXISTS (
->         SELECT *
->         FROM SC
->         WHERE SC.Sno = S.Sno
->         AND SC.Cno = C.Cno
->     )
-> );
+-----+-----+
| Sno | Sname |
+-----+-----+
| 201705001 | 张华 |
+-----+-----+
1 row in set (0.011 sec)
```

图 3-21 例 4.28 查询选修了全部课程的学生学号和姓名

如图 3-21 所示，该步骤展示了“例 4.28 查询选修了全部课程的学生学号和姓名”的执行结果。

-- 例 4.29 查询至少选修了学号为 201715122 学生选修的全部课程的学生学号和姓名

```
mysql> SELECT Sno, Sname
-> FROM Students S
-> WHERE NOT EXISTS (
->     SELECT *
->     FROM SC SC1
->     WHERE SC1.Sno = '201715122'
->     AND NOT EXISTS (
->         SELECT *
->         FROM SC SC2
->         WHERE SC2.Sno = S.Sno
->             AND SC2.Cno = SC1.Cno
->     )
-> );
```

Sno	Sname
201705001	张华
201705002	李玉
201705003	欧阳山
201706001	林艳
201706002	高山
201706003	王海洋

6 rows in set (0.008 sec)

图 3-22 例 4.29 查询至少选修了学号为 201715122 学生选结果

如图 3-22 所示，该步骤展示了“例 4.29 查询至少选修了学号为 201715122 学生选结果”的执行结果。

-- 例 4.30 查询只讲授一门课程的教师姓名和职称

```
mysql> SELECT T.Tname, T.Title
-> FROM Teachers T
-> JOIN Teaches TC ON T.Tno = TC.Tno
-> GROUP BY T.Tno, T.Tname, T.Title
-> HAVING COUNT(DISTINCT TC.Cno) = 1;
```

Tname	Title
张海峰	教授
刘晓明	副教授
王梅	副教授
赵青山	讲师
朱杰	教授
欧阳梅	副教授

6 rows in set (0.015 sec)

图 3-23 例 4.30 查询只讲授一门课程的教师姓名和职称

如图 3-23 所示，该步骤展示了“例 4.30 查询只讲授一门课程的教师姓名和职称”的执行结果。

-- 例 4.31 查询平均成绩高于 85 分的课程号

```
mysql> SELECT Cno, Avg_grade
-> FROM (
->     SELECT Cno, AVG(Grade)
->     FROM SC
->     GROUP BY Cno
-> ) AS Course_avg_grade(Cno, Avg_grade)
-> WHERE Avg_grade > 85;
+-----+-----+
| Cno   | Avg_grade |
+-----+-----+
| CS101 | 86.6667   |
| CS102 | 86.3333   |
| CS201 | 89.0000   |
| CS202 | 85.5000   |
| CS301 | 92.0000   |
+-----+-----+
5 rows in set (0.021 sec)
```

图 3-24 例 4.31 查询平均成绩高于 85 分的课程号

如图 3-24 所示，该步骤展示了“例 4.31 查询平均成绩高于 85 分的课程号”的执行结果。

-- 例 4.32 查询平均成绩最高的课程号和平均成绩

```
mysql> SELECT Cno, MAX(Avg_grade)
-> FROM (
->     SELECT Cno, AVG(Grade) AS Avg_grade
->     FROM SC
->     GROUP BY Cno
-> ) AS Course_avg_grade;
+-----+-----+
| Cno   | MAX(Avg_grade) |
+-----+-----+
| CS101 | 92.0000         |
+-----+-----+
1 row in set (0.010 sec)
```

图 3-25 例 4.32 查询平均成绩最高的课程号和平均成绩

如图 3-25 所示，该步骤展示了“例 4.32 查询平均成绩最高的课程号和平均成绩”的执行结果。

-- 例 4.33 查询选修了 CS301 或 CS306 课程的学生学号

```
mysql> -- 例4.33 查询选修了 CS301 或 CS306 课程的学生学号
Query OK, 0 rows affected (0.004 sec)

mysql> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS301'
-> UNION
-> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS306';
+-----+
| Sno   |
+-----+
| 201705001 |
+-----+
1 row in set (0.008 sec)
```

图 3-26 例 4.33 查询选修了 CS301 或 CS306 课程结果

如图 3-26 所示，该步骤展示了“例 4.33 查询选修了 CS301 或 CS306 课程结果”的执行结果。

-- 例 4.34 查询既选修了 CS301 又选修了 CS306 的学生学号

```
mysql> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS301'
-> INTERSECT
-> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS306';
Empty set (0.071 sec)
```

图 3-27 例 4.34 查询既选修了 CS301 又选修了 CS30 结果

如图 3-27 所示，该步骤展示了“例 4.34 查询既选修了 CS301 又选修了 CS30 结果”的执行结果。

-- 例 4.35 查询选修了 CS301 但未选修 CS306 的学生学号

```
mysql> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS301'
-> EXCEPT
-> SELECT Sno
-> FROM SC
-> WHERE Cno = 'CS306';
+-----+
| Sno   |
+-----+
| 201705001 |
+-----+
1 row in set (0.006 sec)
```

图 3-28 例 4.35 查询选修了 CS301 但未选修 CS306 结果

如图 3-28 所示，该步骤展示了“例 4.35 查询选修了 CS301 但未选修 CS306 结果”的执行结果。

2) 更新操作

根据教材 P94-96 上所有更新实例，完成各类更新操作（插入、修改和删除数据等），请注意标准 SQL 和 OpenGauss 中相应 SQL 语句相同与不同之处。

-- 例 4.36 向 Students 表插入一个学生记录

```
mysql> INSERT INTO Students
-> (Sno, Sname, Sex, Birthday, Speciality, Dno)
-> VALUES
-> ('201716010', '司马相如', '男', '1997-01-28', '计算数学', 'MA
');
Query OK, 1 row affected (0.037 sec)
```

图 3-29 例 4.36 向 Students 表插入一个学生记录

如图 3-29 所示，该步骤展示了“例 4.36 向 Students 表插入一个学生记录”的执行结果。

-- 例 4.37 向 SC 表插入一条选课记录

```
mysql> INSERT INTO SC
-> (Sno, Cno, Grade)
-> VALUES
-> ('201716010', 'MA302', NULL);
Query OK, 1 row affected (0.033 sec)
```

图 3-30 例 4.37 向 SC 表插入一条选课记录

如图 3-30 所示，该步骤展示了“例 4.37 向 SC 表插入一条选课记录”的执行结果。

-- 例 4.38 将信息工程学院教师办理就餐卡

```
mysql> INSERT INTO Cardinf
-> (Card_no, Name, Balance)
-> SELECT Tno, Tname, 100.00
-> FROM Teachers
-> WHERE Dno = 'MA';
Query OK, 2 rows affected (0.088 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

图 3-31 例 4.38 将信息工程学院教师办理就餐卡

如图 3-31 所示，该步骤展示了“例 4.38 将信息工程学院教师办理就餐卡”的执行结果。

-- 例 4.39 删除学号为 201624010 的学生记录

```
mysql> DELETE FROM Students
-> WHERE Sno = '201716010';
Query OK, 1 row affected (0.029 sec)
```

图 3-32 例 4.39 删除学号为 201624010 的学生记录

如图 3-32 所示，该步骤展示了“例 4.39 删除学号为 201624010 的学生记录”的执行结果。

-- 例 4.40 删除计算数学专业所有学生的选课记录

```
mysql> DELETE FROM SC
-> WHERE Sno IN (
->     SELECT Sno
->     FROM Students
->     WHERE Speciality = '计算数学'
-> );
Query OK, 0 rows affected (0.055 sec)
```

图 3-33 例 4.40 删除计算数学专业所有学生的选课记录

如图 3-33 所示，该步骤展示了“例 4.40 删除计算数学专业所有学生的选课记录”的执行结果。

-- 例 4.41 将职工号为 B050041 的教师职称修改为副教授

```
mysql> UPDATE Teachers
-> SET Title = '副教授'
-> WHERE Tno = '05004';
Query OK, 1 row affected (0.045 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

图 3-34 例 4.41 将职工号为 B050041 的教师职称修改为结果

如图 3-34 所示，该步骤展示了“例 4.41 将职工号为 B050041 的教师职称修改为结果”的执行结果。

-- 例 4.42 将软件工程课程成绩低于 60 分的学生成绩提高 5 分

```
mysql> UPDATE SC
-> SET Grade = Grade + 5
-> WHERE Grade < 60
-> AND Cno IN (
->     SELECT Cno
->     FROM Courses
->     WHERE Cname = '计算数学'
-> );
Query OK, 0 rows affected (0.019 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

图 3-35 例 4.42 将软件工程课程成绩低于 60 分的学生成绩提高 5 结果

如图 3-35 所示，该步骤展示了“例 4.42 将软件工程课程成绩低于 60 分的学生成绩提高 5 结果”的执行结果。

3.实验分析与总结

本次实验完成了学生选课数据库的查询与更新操作，主要涉及 Students、Courses、Teachers、Departments 和 SC 等数据表。通过实验，练习了简单查询、条件查询、模糊查询、多表连接查询、分组统计查询以及数据插入、修改和删除等基本 SQL 操作，进一步理解了关系数据库中表之间的联系及 SQL 语句的实际应用。在实验过程中，遇

到了字段数量不匹配、字段长度超限以及外键约束导致删除失败等问题，通过检查表结构和分析报错信息，对 SQL 语句进行了相应调整，提升了实际数据库操作与问题排查能力。

通过本次实验，较为系统地掌握了常用的 SQL 查询与更新语句的使用方法，加深了对数据库基本操作、数据完整性约束以及多表关联机制的理解。同时也认识到教材中的标准 SQL 示例在实际数据库环境中可能需要根据具体表结构进行适当修改，不能直接照搬使用。本次实验不仅巩固了数据库理论知识，也提高了独立分析问题、解决问题和实际动手操作的能力，为后续数据库设计与开发实验奠定了良好的基础。

问题及解决办法

问题：在分组查询中误将聚集函数写入 WHERE 条件，导致 SQL 语句报错。

解决办法：将聚集函数筛选条件改写为 HAVING 子句中，并重新检查 GROUP BY 语句结构。

心得体会：通过本实验，我进一步熟悉了查询、连接、分组统计和更新操作的实际用法。

实验 4 视图练习

1. 实验目的

1.) 熟悉通过 SQL 对视图进行操作。

2) 体会视图的作用。

2. 实验内容

1) 使用 SQL 语句创建视图。

[1] 创建基于单个基本表的视图

创建“计算机专业学生”的视图，只保留 Students 表中除 Speciality 外的部分字段。

```
mysql> CREATE OR REPLACE VIEW CS_Students
-> AS
-> SELECT Sno, Sname, Sex, Birthday, Dno
-> FROM Students
-> WHERE Speciality = '计算机';
Query OK, 0 rows affected (0.207 sec)

mysql> SELECT *
-> FROM CS_Students;
+-----+-----+-----+-----+-----+
| Sno   | Sname | Sex  | Birthday | Dno |
+-----+-----+-----+-----+-----+
| 201705001 | 张华 | 男   | 1999-12-16 | CS |
| 201705002 | 李玉 | 女   | 1998-10-17 | CS |
| 201705003 | 欧阳山 | 男   | 1999-05-24 | CS |
+-----+-----+-----+-----+-----+
3 rows in set (0.122 sec)
```

图 4-1 创建“计算机专业学生”的视图，只保留 Students 结果

如图 4-1 所示，该步骤展示了“创建“计算机专业学生”的视图，只保留 Students 结果”的执行结果。

[2] 创建基于子查询的视图

创建计算机系学生选课视图 CS_SC。

```
mysql> CREATE OR REPLACE VIEW CS_SC(Sno, Cno, Grade)
-> AS
-> SELECT *
-> FROM SC
-> WHERE Sno IN (
->   SELECT Sno
->   FROM Students
->   WHERE Dno = 'CS'
-> );
Query OK, 0 rows affected (0.391 sec)

mysql> SELECT *
-> FROM CS_SC;
+-----+-----+-----+
| Sno   | Cno   | Grade |
+-----+-----+-----+
| 201705001 | CS101 | 89 |
| 201705001 | CS102 | 90 |
| 201705001 | CS201 | 88 |
| 201705001 | CS202 | 86 |
| 201705001 | CS301 | 92 |
| 201705002 | CS101 | 91 |
| 201705002 | CS102 | 87 |
| 201705002 | CS201 | 89 |
| 201705002 | CS202 | 85 |
| 201705003 | CS101 | 80 |
| 201705003 | CS102 | 82 |
| 201705003 | CS201 | 90 |
+-----+-----+-----+
12 rows in set (0.131 sec)
```

图 4-2 创建计算机系学生选课视图 CS_SC

如图 4-2 所示，该步骤展示了“创建计算机系学生选课视图 CS_SC”的执行结果。

[3] 创建多表连接视图

创建学生成绩视图 Student_Grades，包含学号、姓名、课程名、成绩。

```
mysql> CREATE OR REPLACE VIEW Student_Grades(Sno, Sname, Cname, Grade)
-> AS
-> SELECT S.Sno, Sname, Cname, Grade
-> FROM Students S, SC, Courses C
-> WHERE S.Sno = SC.Sno
->       AND C.Cno = SC.Cno;
Query OK, 0 rows affected (0.067 sec)

mysql> SELECT *
-> FROM Student_Grades;
```

Sno	Sname	Cname	Grade
201705001	张华	计算科学导论	89
201705002	李玉	计算科学导论	91
201705003	欧阳山	计算科学导论	80
201705001	张华	C语言	90
201705002	李玉	C语言	87
201705003	欧阳山	C语言	82
201705001	张华	数据结构	88
201705002	李玉	数据结构	89
201705003	欧阳山	数据结构	90
201705001	张华	离散数学	86
201705002	李玉	离散数学	85
201705001	张华	数据库系统原理	92

12 rows in set (0.116 sec)

图 4-3 创建学生成绩视图 Student_Grades，包含学号结果

如图 4-3 所示，该步骤展示了“创建学生成绩视图 Student_Grades，包含学号结果”的执行结果。

[4]. 基于基本表和视图创建新视图

创建计算机系所有学生成绩视图 CS_Student_Grades。

```
mysql> CREATE OR REPLACE VIEW CS_Student_Grades(Sno, Sname, Cname, Grade)
-> AS
-> SELECT S.Sno, S.Sname, Cname, Grade
-> FROM Students S, Student_Grades SG
-> WHERE S.Sno = SG.Sno
->       AND Speciality = '计算机';
Query OK, 0 rows affected (0.067 sec)

mysql> SELECT *
-> FROM CS_Student_Grades;
```

Sno	Sname	Cname	Grade
201705001	张华	计算科学导论	89
201705001	张华	C语言	90
201705001	张华	数据结构	88
201705001	张华	离散数学	86
201705001	张华	数据库系统原理	92
201705002	李玉	计算科学导论	91
201705002	李玉	C语言	87
201705002	李玉	数据结构	89
201705002	李玉	离散数学	85
201705003	欧阳山	计算科学导论	80
201705003	欧阳山	C语言	82
201705003	欧阳山	数据结构	90

12 rows in set (0.063 sec)

图 4-4 创建计算机系所有学生成绩视图 CS_Student_Gr 结果

如图 4-4 所示，该步骤展示了“创建计算机系所有学生成绩视图 CS_Student_Gr 结

果”的执行结果。

[5]. 创建聚合视图

创建学生平均成绩视图 Student_Avg_Grades。

```
mysql> CREATE OR REPLACE VIEW Student_Avg_Grades(Sno, Sname, Avg_Grade)
-> AS
-> SELECT S.Sno, Sname, AVG(Grade)
-> FROM Students S, SC
-> WHERE S.Sno = SC.Sno
-> GROUP BY S.Sno, Sname;
Query OK, 0 rows affected (0.378 sec)

mysql> SELECT *
-> FROM Student_Avg_Grades;
+-----+-----+-----+
| Sno      | Sname  | Avg_Grade |
+-----+-----+-----+
| 201705001 | 张华   | 89.0000   |
| 201705002 | 李玉   | 88.0000   |
| 201705003 | 欧阳山 | 84.0000   |
+-----+-----+-----+
3 rows in set (0.394 sec)
```

图 4-5 创建学生平均成绩视图 Student_Avg_Grade 结果

如图 4-5 所示，该步骤展示了“创建学生平均成绩视图 Student_Avg_Grade 结果”的执行结果。

2) 对视图执行创建、查找、更新等操作，对比和基本表进行查找、更新操作有无不同之处。

删除视图操作

[1]. 删除普通视图

```
mysql> DROP VIEW CS_Students;
Query OK, 0 rows affected (0.347 sec)
```

图 4-6 [1]. 删除普通视图

如图 4-6 所示，该步骤展示了“[1]. 删除普通视图”的执行结果。

[2]. 使用 RESTRICT 删除视图

```
mysql> DROP VIEW Student_Grades RESTRICT;
Query OK, 0 rows affected (0.405 sec)
```

图 4-7 [2]. 使用 RESTRICT 删除视图

如图 4-7 所示，该步骤展示了“[2]. 使用 RESTRICT 删除视图”的执行结果。

[3]. 使用 CASCADE 删除视图


```
mysql> DROP VIEW Student_Grades CASCADE;
Query OK, 0 rows affected (0.384 sec)
```

图 4-8 [3]. 使用 CASCADE 删除视图

如图 4-8 所示，该步骤展示了“[3]. 使用 CASCADE 删除视图”的执行结果。

基于视图的查询

[1]. 查询计算机专业男生

```
mysql> SELECT *
-> FROM CS_Students
-> WHERE Sex = '男';
```

Sno	Sname	Sex	Birthday	Dno
201705001	张华	男	1999-12-16	CS
201705003	欧阳山	男	1999-05-24	CS

```
2 rows in set (0.026 sec)
```

图 4-9 [1]. 查询计算机专业男生

如图 4-9 所示，该步骤展示了“[1]. 查询计算机专业男生”的执行结果。

[2]. 查询某个学生的各科成绩

```
mysql> SELECT Sname, Cname, Grade
-> FROM Student_Grades
-> WHERE Sno = '202405001';
Empty set (0.034 sec)
```

图 4-10 [2]. 查询某个学生的各科成绩

如图 4-10 所示，该步骤展示了“[2]. 查询某个学生的各科成绩”的执行结果。

更新视图操作

[1]. 创建物化视图

```
mysql> CREATE MATERIALIZED VIEW MA_Students(Sno, Sname, Sex)
-> AS
-> SELECT Sno, Sname, Sex
-> FROM Students
-> WHERE Speciality = '数学';
Query OK, 0 rows affected (0.377 sec)

mysql> SELECT COUNT(*)
-> FROM MA_Students;
```

COUNT(*)
3

```
1 row in set, 1 warning (0.065 sec)
```

图 4-11 [1]. 创建物化视图

如图 4-11 所示，该步骤展示了“[1]. 创建物化视图”的执行结果。

[2]. 向基本表插入新数据

```
mysql> INSERT INTO Students
-> (Sno, Sname, Sex, Birthday, Speciality, Dno)
-> VALUES
-> ('202406004', '张三', '男', '2006-05-19', '数学', 'MA'),
-> ('202406005', '李四', '女', '2006-05-19', '数学', 'MA'),
-> ('202406006', '王五', '男', '2006-05-19', '数学', 'MA'),
-> ('202406007', '张芳', '男', '2006-05-19', '数学', 'MA'),
-> ('202406008', '李峰', '男', '2006-05-19', '数学', 'MA');
Query OK, 5 rows affected (0.672 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

图 4-12 [2]. 向基本表插入新数据

如图 4-12 所示，该步骤展示了“[2]. 向基本表插入新数据”的执行结果。

[3]. 手动刷新物化视图

```
mysql> SELECT COUNT(*)
-> FROM MA_Students;
+-----+
| COUNT(*) |
+-----+
|          8 |
+-----+
1 row in set, 1 warning (0.138 sec)
```

图 4-13 [3]. 手动刷新物化视图

如图 4-13 所示，该步骤展示了“[3]. 手动刷新物化视图”的执行结果。

[4]. 删除刚插入的测试数据

```
mysql> DELETE FROM Students
-> WHERE Sno BETWEEN '202406004' AND '202406008';
Query OK, 5 rows affected (0.412 sec)
```

图 4-14 [4]. 删除刚插入的测试数据

如图 4-14 所示，该步骤展示了“[4]. 删除刚插入的测试数据”的执行结果。

3.实验分析与总结

本次实验主要练习了视图的创建、查询、删除以及物化视图的刷新操作。通过实验可以看出，视图本质上是基于一个或多个基本表定义的虚拟表，本身通常不单独存储数据，查询视图时系统会根据视图定义转换为对基本表的查询。使用视图可以简化多表连接查询，例如 Student_Grades 视图将学生表、选课表和课程表连接起来，后续查询学

生成成绩时不需要每次都重复书写复杂的连接条件，提高了 SQL 语句的可读性和复用性。

同时，实验也体现了普通视图和物化视图的区别。普通视图的数据会随着基本表查询结果动态变化，而物化视图会将查询结果实际保存下来，因此当基本表数据发生变化后，物化视图不会自动同步，需要使用 `REFRESH MATERIALIZED VIEW` 手动刷新。删除视图时，如果该视图被其他视图依赖，直接删除或使用 `RESTRICT` 会失败，而使用 `CASCADE` 可以同时删除依赖该视图的其他对象。通过本次实验，我进一步理解了视图在数据库中的封装、简化查询和数据安全控制作用，也掌握了视图和基本表在查询、删除及数据同步方面的差异。

问题及解决办法

问题：删除存在依赖关系的视图时，直接删除或使用 `RESTRICT` 可能失败。

解决办法：先检查视图之间的依赖关系，必要时使用 `CASCADE` 级联删除，并在基本表变化后及时刷新物化视图。

心得体会：本实验帮助我理解了视图在简化查询、封装数据和控制访问方面的作用。

实验 5 使用 JDBC 连接数据库

1. 实验目的

了解如何使用 JDBC 连接数据库。JDBC (Java Database Connectivity, Java 数据库连接) 是一种用于执行 SQL 语句的 Java API, 可以为多种关系数据库提供统一访问接口, 应用程序可基于它操作数据。OpenGauss 库提供了对 JDBC 4.0 特性的支持, 需要使用 JDK1.8 版本编译程序代码。

2. 实验内容

1) 在 OpenGauss 中创建数据库、表;

创建数据库:

```
mysql> CREATE DATABASE IF NOT EXISTS test_db
      -> DEFAULT CHARACTER SET utf8mb4
      -> DEFAULT COLLATE utf8mb4_unicode_ci;
Query OK, 1 row affected, 1 warning (0.045 sec)

mysql> USE test_db;
Database changed
```

图 5-1 创建数据库

如图 5-1 所示, 该步骤展示了“创建数据库”的执行结果。

创建表:

```
mysql> CREATE TABLE IF NOT EXISTS test_tb (
      ->   id INT PRIMARY KEY,
      ->   name CHAR(20),
      ->   sex CHAR(3),
      ->   CHECK (sex IN ('男', '女'))
      -> );
Query OK, 0 rows affected, 1 warning (0.177 sec)

mysql> DESC test_tb;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int           | NO   | PRI | NULL    |       |
| name  | varchar(20)   | YES  |     | NULL    |       |
| sex   | varchar(3)    | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.108 sec)
```

图 5-2 创建表

如图 5-2 所示, 该步骤展示了“创建表”的执行结果。

2) 使用 JDBC 连接到新创建的数据库;

```

public class Connect0Insert {
    public static void main(String[] args) {
        Connection con = null;
        PreparedStatement pstmt = null;

        String driver = "com.mysql.cj.jdbc.Driver";

        String url = "jdbc:mysql://localhost:3306/test_db"
            + "?useSSL=false"
            + "&allowPublicKeyRetrieval=true"
            + "&serverTimezone=Asia/Shanghai"
            + "&useUnicode=true"
            + "&characterEncoding=utf8";
    }
}

```

图 5-3 2) 使用 JDBC 连接到新创建的数据库

如图 5-3 所示，该步骤展示了“2) 使用 JDBC 连接到新创建的数据库”的执行结果。

连接成功:

```
MySQL connect success!
```

图 5-4 连接成功

如图 5-4 所示，该步骤展示了“连接成功”的执行结果。

3) 在 Java 程序中改变数据库中的值或者输出数据库中的值;

使用 JDBC 进行插值

```

String sql = "INSERT INTO test_tb(id, name, sex) VALUES (?, ?, ?) "
    + "ON DUPLICATE KEY UPDATE name = VALUES(name), sex = VALUES(sex)";

```

图 5-5 使用 JDBC 进行插值

如图 5-5 所示，该步骤展示了“使用 JDBC 进行插值”的执行结果。

插值成功:

```

D:\Tools\java\jdk\bin\java.exe ...
MySQL connect success!
Insert or update success!
Affected rows: 1

```

```
进程已结束，退出代码为 0
```

图 5-6 插值成功

如图 5-6 所示，该步骤展示了“插值成功”的执行结果。

4) 请参考华为提供的实验手册：**OpenGauss 场景化综合应用实验**。

3.实验分析与总结

本次实验主要学习了如何使用 JDBC 在 Java 程序中连接数据库，并通过 Java 代码执行 SQL 语句。JDBC 为 Java 程序访问关系型数据库提供了统一接口，程序通过加载数据库驱动、建立连接、创建 SQL 执行对象、执行 SQL 语句和关闭资源等步骤，实现了对数据库的基本操作。通过本次实验，我掌握了 JDBC 连接数据库的基本流程，也理解了 Connection、Statement 和 ResultSet 等对象在数据库访问过程中的作用。

由于视频教程使用的是 openGauss 数据库，而本次实验环境为 MySQL，因此部分语法和驱动配置需要进行转换。例如，openGauss 使用 org.postgresql.Driver 驱动和 jdbc:postgresql://... 连接地址，而 MySQL 需要使用 com.mysql.cj.jdbc.Driver 驱动和 jdbc:mysql://... 连接地址。通过对比两种数据库的差异，本次实验不仅完成了 JDBC 连接和操作数据库的任务，也加深了对不同数据库 SQL 语法差异和 JDBC 通用性的理解

问题及解决办法

问题：JDBC 连接时容易因驱动名称、连接地址或端口配置错误导致连接失败。

解决办法：检查数据库驱动、URL 格式、用户名和密码，并确认数据库服务正常运行后重新连接。

心得体会：本实验使我掌握了 Java 程序通过 JDBC 访问数据库的基本流程。

实验 6 完整性控制

1. 实验目的

- 1) 了解 OpenGauss 提供的完整性机制，熟悉通过 SQL 对数据进行完整性控制。
- 2) 完成教材第五章中的例子。

2. 实验内容

1) 练习创建下面约束：

主键（PRIMARY KEY）约束；

2.1.1 创建表时直接指定主键

```
mysql> CREATE TABLE test_pk (  
->     id INT PRIMARY KEY,  
->     name VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.226 sec)
```

图 6-1 2.1.1 创建表时直接指定主键

如图 6-1 所示，该步骤展示了“2.1.1 创建表时直接指定主键”的执行结果。

测试主键唯一性：

```
mysql> INSERT INTO test_pk VALUES (1, '张三');  
Query OK, 1 row affected (0.241 sec)  
  
mysql> INSERT INTO test_pk VALUES (1, '李四');  
ERROR 1062 (23000): Duplicate entry '1' for key 'test_pk.PRIM  
ARY'
```

图 6-2 测试主键唯一性

如图 6-2 所示，该步骤展示了“测试主键唯一性”的执行结果。

第二条会报错，因为 id 是主键，不能重复。

2.1.2 创建表后添加主键

```
mysql> CREATE TABLE test_pk2 (
->     id INT,
->     name VARCHAR(20)
-> );
Query OK, 0 rows affected (0.215 sec)

mysql>
mysql> ALTER TABLE test_pk2
-> ADD PRIMARY KEY (id);
Query OK, 0 rows affected (0.876 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

图 6-3 2.1.2 创建表后添加主键

如图 6-3 所示，该步骤展示了“2.1.2 创建表后添加主键”的执行结果。

2.1.3 联合主键

```
mysql> CREATE TABLE test_sc (
->     sno INT,
->     cno INT,
->     grade SMALLINT CHECK (grade >= 0 AND grade <= 100)
,
->     PRIMARY KEY (sno, cno)
-> );
Query OK, 0 rows affected (0.211 sec)
```

图 6-4 2.1.3 联合主键

如图 6-4 所示，该步骤展示了“2.1.3 联合主键”的执行结果。

测试成绩检查约束：

```
mysql> INSERT INTO test_sc VALUES (1, 1, 90);
Query OK, 1 row affected (0.353 sec)

mysql> INSERT INTO test_sc VALUES (1, 2, 120);
ERROR 3819 (HY000): Check constraint 'test_sc_chk_1' is violated.
mysql>
```

图 6-5 测试成绩检查约束

如图 6-5 所示，该步骤展示了“测试成绩检查约束”的执行结果。

第二条会报错，因为 grade 超出了 0~100 范围

唯一性（UNIQUE）约束：

2.1.4 创建课程表时设置课程名唯一


```
mysql>
mysql> CREATE TABLE test_courses (
->     cno CHAR(5) PRIMARY KEY,
->     cname CHAR(20) UNIQUE,
->     period SMALLINT,
->     credit SMALLINT
-> );
Query OK, 0 rows affected (0.536 sec)
```

图 6-6 2.1.4 创建课程表时设置课程名唯一

如图 6-6 所示，该步骤展示了“2.1.4 创建课程表时设置课程名唯一”的执行结果。

测试唯一性约束：

```
mysql> INSERT INTO test_courses VALUES ('C001', '数据库', 64, 4);
Query OK, 1 row affected (0.344 sec)

mysql> INSERT INTO test_courses VALUES ('C002', '数据库', 48, 3);
ERROR 1062 (23000): Duplicate entry '数据库' for key 'test_courses.cname'
```

图 6-7 测试唯一性约束

如图 6-7 所示，该步骤展示了“测试唯一性约束”的执行结果。

第二条会报错，因为 cname 被设置为唯一，不能重复。

将 STUDENTS 表中的 SNAME 列设为唯一键约束、COURSES 表中的 CNAME 列设为唯一键约束；

2.1.5 给已有 Students 表的 Sname 添加唯一约束

```
mysql> ALTER TABLE Students
-> ADD CONSTRAINT uq_students_sname UNIQUE (Sname);
```

图 6-8 2.1.5 给已有 Students 表的 Sname 结果

如图 6-8 所示，该步骤展示了“2.1.5 给已有 Students 表的 Sname 结果”的执行结果。

2.1.6 给已有 Courses 表的 Cname 添加唯一约束

```
mysql> ALTER TABLE Students
-> ADD CONSTRAINT uq_students_sname UNIQUE (Sname);
```

图 6-9 2.1.6 给已有 Courses 表的 Cname 添结果

如图 6-9 所示，该步骤展示了“2.1.6 给已有 Courses 表的 Cname 添结果”的执行

结果。

2.1.7 非空约束 NOT NULL

测试非空约束：

```
mysql> INSERT INTO test_notnull VALUES (1, NULL);  
ERROR 1048 (23000): Column 'col1' cannot be null
```

图 6-10 测试非空约束

如图 6-10 所示，该步骤展示了“测试非空约束”的执行结果。

会报错，因为 col1 已经被设置为 NOT NULL。

检查（CHECK）约束等；

2.1.8 插入符合约束的数据：

```
mysql> INSERT INTO EmpSalary  
-> VALUES ('E0001', '张三', 'CS', 5000, 500, 500, 100, 100, 400);  
Query OK, 1 row affected (0.360 sec)
```

图 6-11 2.1.8 插入符合约束的数据

如图 6-11 所示，该步骤展示了“2.1.8 插入符合约束的数据”的执行结果。

插入不符合约束的数据：

```
mysql> INSERT INTO EmpSalary  
-> VALUES ('E0002', '李四', 'CS', 5000, 100, 100, 1000, 1000, 100  
0);  
ERROR 3819 (HY000): Check constraint 'check_salary' is violated.
```

图 6-12 插入不符合约束的数据

如图 6-12 所示，该步骤展示了“插入不符合约束的数据”的执行结果。

会报错，因为实际收入小于基本工资的一半。

外键(FOREIGN KEY)约束：外键约束是为了强制实现表之间的参照完整性。级联参照完整性约束就是为了保证外键数据的关联性。

2.1.9 创建部门表和教师表

```

mysql> CREATE TABLE Teachers (
->     Tno CHAR(7) PRIMARY KEY,
->     Tname CHAR(10) NOT NULL,
->     Sex CHAR(3) CHECK (Sex = '男' OR Sex = '女'),
->     Birthday DATE,
->     Title VARCHAR(15),
->     Dno CHAR(4),
->     CONSTRAINT fk_teachers_dno
->         FOREIGN KEY (Dno)
->         REFERENCES Departments(Dno)
-> );
Query OK, 0 rows affected (0.874 sec)

mysql> INSERT INTO Departments VALUES
-> ('CS', '计算机系', NULL),
-> ('MA', '数学系', NULL);
Query OK, 2 rows affected (0.237 sec)

```

图 6-13 2.1.9 创建部门表和教师表

如图 6-13 所示，该步骤展示了“2.1.9 创建部门表和教师表”的执行结果。

插入测试数据

```

mysql> INSERT INTO Departments VALUES
-> ('CS', '计算机系', NULL),
-> ('MA', '数学系', NULL);
Query OK, 2 rows affected (0.237 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Teachers VALUES
-> ('T0001', '张老师', '男', '1980-01-01', '教授', 'CS'),
-> ('T0002', '李老师', '女', '1985-05-10', '副教授', 'MA');
Query OK, 2 rows affected (0.071 sec)

```

图 6-14 插入测试数据

如图 6-14 所示，该步骤展示了“插入测试数据”的执行结果。

测试外键约束

```

mysql> INSERT INTO Teachers VALUES
-> ('T0003', '王老师', '男', '1990-03-15', '讲师', 'XX');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key c
onstraint fails (`test_db`.`teachers`, CONSTRAINT `fk_teachers_dno` F
OREIGN KEY (`Dno`) REFERENCES `departments` (`Dno`))

```

图 6-15 测试外键约束

如图 6-15 所示，该步骤展示了“测试外键约束”的执行结果。

会报错，因为 XX 在 Departments 表中不存在。

2.1.10 级联参照完整性约束

删除原外键

```
mysql> ALTER TABLE Teachers
-> DROP FOREIGN KEY fk_teachers_dno;
Query OK, 0 rows affected (0.492 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

图 6-16 删除原外键

如图 6-16 所示，该步骤展示了“删除原外键”的执行结果。

添加级联外键

```
Database changed
mysql> ALTER TABLE Teachers
-> ADD CONSTRAINT fk_teachers_dno
-> FOREIGN KEY (Dno)
-> REFERENCES Departments(Dno)
-> ON DELETE CASCADE
-> ON UPDATE CASCADE;
Query OK, 2 rows affected (0.926 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

图 6-17 添加级联外键

如图 6-17 所示，该步骤展示了“添加级联外键”的执行结果。

测试 ON UPDATE CASCADE

```
mysql> UPDATE Departments
-> SET Dno = 'MATH'
-> WHERE Dno = 'MA';
Query OK, 1 row affected (0.390 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> SELECT *
-> FROM Teachers;
+-----+-----+-----+-----+-----+-----+
| Tno   | Tname   | Sex   | Birthday   | Title   | Dno   |
+-----+-----+-----+-----+-----+-----+
| T0001 | 张老师   | 男    | 1980-01-01 | 教授    | CS    |
| T0002 | 李老师   | 女    | 1985-05-10 | 副教授  | MATH  |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.013 sec)
```

图 6-18 测试 ON UPDATE CASCADE

如图 6-18 所示，该步骤展示了“测试 ON UPDATE CASCADE”的执行结果。

Departments 表中 MA 改成 MATH 后，Teachers 表中引用该部门号的记录也会自动更新。

测试 ON DELETE CASCADE

```
mysql> DELETE FROM Departments
      -> WHERE Dno = 'MATH';
Query OK, 1 row affected (0.049 sec)

mysql>
mysql> SELECT *
      -> FROM Teachers;
+-----+-----+-----+-----+-----+-----+
| Tno   | Tname  | Sex   | Birthday   | Title  | Dno   |
+-----+-----+-----+-----+-----+-----+
| T0001 | 张老师 | 男    | 1980-01-01 | 教授   | CS    |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.013 sec)
```

图 6-19 测试 ON DELETE CASCADE

如图 6-19 所示，该步骤展示了“测试 ON DELETE CASCADE”的执行结果。

删除 Departments 中的 MATH 部门后，Teachers 表中属于该部门的教师记录也会被自动删除。

2) 创建触发器

触发器是一种特殊类型的存储过程，通常用于实现强制业务规则和数据完整性。创建 UPDATE 触发器检查修改操作的业务规则，如：在 STUDENTS 表上建立触发器，当用户修改学生的学号时给出提示信息。

2.2.1 创建触发器：禁止修改学生学号

```
mysql> CREATE TRIGGER trg_students_sno_update
      -> BEFORE UPDATE ON Students
      -> FOR EACH ROW
      -> BEGIN
      ->     IF OLD.Sno <> NEW.Sno THEN
      ->         SIGNAL SQLSTATE '45000'
      ->         SET MESSAGE_TEXT = '提示：学号不允许修改！';
      ->     END IF;
      -> END //
Query OK, 0 rows affected (0.112 sec)
```

图 6-20 2.2.1 创建触发器：禁止修改学生学号

如图 6-20 所示，该步骤展示了“2.2.1 创建触发器：禁止修改学生学号”的执行结果。

测试触发器

```
mysql> UPDATE Students
      -> SET Sno = '202406001'
      -> WHERE Sno = '201705001';
ERROR 1644 (45000): 提示：学号不允许修改！
mysql> |
```

图 6-21 测试触发器

如图 6-21 所示，该步骤展示了“测试触发器”的执行结果。

3. 实验分析与总结

分析：

本次实验主要练习了数据库完整性控制机制，包括主键约束、唯一性约束、非空约束、检查约束、外键约束以及触发器。通过主键约束可以保证表中每条记录具有唯一标识；通过唯一性约束可以保证某些字段的值不重复，例如学生姓名或课程名；通过非空约束可以限制关键字段不能为空；通过检查约束可以限定字段取值范围，例如成绩必须在 0 到 100 之间；通过外键约束可以维护不同表之间的参照完整性，防止出现无效引用。实验中还练习了级联更新和级联删除，进一步理解了父表数据变化时对子表数据的影响。

总结：

通过本次实验，我掌握了使用 SQL 语句创建和测试多种完整性约束的方法，也理解了完整性控制在数据库设计中的重要作用。完整性约束能够有效保证数据库中数据的正确性、一致性和可靠性。

问题及解决办法

问题：插入或更新数据时，可能因违反主键、外键或 CHECK 约束导致操作失败。

解决办法：根据报错信息检查数据是否满足完整性约束，修改不合法数据或调整表结构后重新执行。

心得体会：本实验使我认识到完整性约束对保证数据正确性和一致性具有重要作用。

实验 7 安全性控制

1. 实验目的

- 1) 了解 OpenGauss 提供的安全性手段。
- 2) 熟悉通过 SQL 对数据进行安全性控制。
- 3) 领会书上讲解的各种安全性方法。
- 4) 重点掌握授权和回收授权方法。

2. 实验内容

- 1) 创建一些数据库用户，并理解数据库用户和角色之间的关系。
- 2) 利用查询分析器熟悉通过 SQL 对数据进行安全性控制 (grant 和 revoke 语句)。

7.1 创建测试用户

```
mysql> CREATE USER 'tom'@'localhost'  
-> IDENTIFIED BY 'OpenGauss@2024';  
Query OK, 0 rows affected (0.372 sec)
```

图 7-1 7.1 创建测试用户

如图 7-1 所示，该步骤展示了“7.1 创建测试用户”的执行结果。

查看用户是否创建成功：

```
mysql> SELECT user, host  
-> FROM mysql.user  
-> WHERE user = 'tom';  
+-----+-----+  
| user | host      |  
+-----+-----+  
| tom  | localhost |  
+-----+-----+  
1 row in set (0.020 sec)
```

图 7-2 查看用户是否创建成功

如图 7-2 所示，该步骤展示了“查看用户是否创建成功”的执行结果。

7.2 创建角色

```
mysql> CREATE ROLE 'jim';  
Query OK, 0 rows affected (0.041 sec)
```

图 7-3 7.2 创建角色

如图 7-3 所示，该步骤展示了“7.2 创建角色”的执行结果。

查看角色：

```
mysql> SELECT user, host
-> FROM mysql.user
-> WHERE user = 'jim';
+-----+-----+
| user | host |
+-----+-----+
| jim  | %    |
+-----+-----+
1 row in set (0.298 sec)
```

图 7-4 查看角色

如图 7-4 所示，该步骤展示了“查看角色”的执行结果。

7.3 将角色授权给用户

```
mysql> GRANT 'jim' TO 'tom'@'localhost';
Query OK, 0 rows affected (0.347 sec)

mysql>
mysql> SET DEFAULT ROLE 'jim' TO 'tom'@'localhost';
Query OK, 0 rows affected (0.036 sec)
```

图 7-5 7.3 将角色授权给用户

如图 7-5 所示，该步骤展示了“7.3 将角色授权给用户”的执行结果。

查看 tom 的权限：

```
mysql> SHOW GRANTS FOR 'tom'@'localhost';
+-----+
| Grants for tom@localhost |
+-----+
| GRANT USAGE ON *.* TO `tom`@`localhost` |
| GRANT `jim`@`%` TO `tom`@`localhost` |
+-----+
2 rows in set (0.214 sec)
```

图 7-6 查看 tom 的权限

如图 7-6 所示，该步骤展示了“查看 tom 的权限”的执行结果。

7.4 给用户授予数据库全部权限

```
mysql> GRANT ALL PRIVILEGES ON qyb.* TO 'tom'@'localhost';
Query OK, 0 rows affected (0.322 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected, 1 warning (0.047 sec)
```

图 7-7 给用户授予数据库全部权限

如图 7-7 所示，该步骤展示了“给用户授予数据库全部权限”的执行结果。

查看授权结果：

```
mysql> SHOW GRANTS FOR 'tom'@'localhost';
+-----+
| Grants for tom@localhost |
+-----+
| GRANT USAGE ON *.* TO `tom`@`localhost` |
| GRANT ALL PRIVILEGES ON `qyb`.* TO `tom`@`localhost` |
| GRANT `jim`@`%` TO `tom`@`localhost` |
+-----+
```

图 7-8 查看授权结果

如图 7-8 所示，该步骤展示了“查看授权结果”的执行结果。

7.5 回收用户全部权限

```
mysql> REVOKE ALL PRIVILEGES, GRANT OPTION
-> FROM 'tom'@'localhost';
Query OK, 0 rows affected (0.335 sec)
```

图 7-9 回收用户全部权限

如图 7-9 所示，该步骤展示了“回收用户全部权限”的执行结果。

再次查看权限：

```
mysql> SHOW GRANTS FOR 'tom'@'localhost';
+-----+
| Grants for tom@localhost |
+-----+
| GRANT USAGE ON *.* TO `tom`@`localhost` |
| GRANT `jim`@`%` TO `tom`@`localhost` |
+-----+
2 rows in set (0.008 sec)
```

图 7-10 再次查看权限

如图 7-10 所示，该步骤展示了“再次查看权限”的执行结果。

7.6 授予用户对表的查询权限

```
mysql> GRANT SELECT
-> ON qyb.Teachers
-> TO 'tom'@'localhost';
Query OK, 0 rows affected (0.170 sec)
```

图 7-11 7.6 授予用户对表的查询权限

如图 7-11 所示，该步骤展示了“7.6 授予用户对表的查询权限”的执行结果。

可以用 tom 登录 MySQL:

```
C:\Users\qiyin>mysql -u tom -pOpenGauss@2024
mysql: [Warning] Using a password on the command line interface
is insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 20
Server version: 9.7.0 MySQL Community Server - GPL
```

图 7-12 可以用 tom 登录 MySQL

如图 7-12 所示，该步骤展示了“可以用 tom 登录 MySQL”的执行结果。

7.7 回收用户对表的查询权限

```
mysql> REVOKE SELECT
-> ON qyb.Teachers
-> FROM 'tom'@'localhost';
Query OK, 0 rows affected (0.353 sec)
```

图 7-13 7.7 回收用户对表的查询权限

如图 7-13 所示，该步骤展示了“7.7 回收用户对表的查询权限”的执行结果。

再次用 tom 查询:

```
mysql> SELECT *
-> FROM Teachers;
ERROR 1046 (3D000): No database selected
```

图 7-14 再次用 tom 查询

如图 7-14 所示，该步骤展示了“再次用 tom 查询”的执行结果。

提示权限不足。

7.8 授予用户对指定列的权限

授予 tom 查询 Sno、Cno 的权限，以及修改 Grade 的权限：

```
mysql> GRANT SELECT (Sno, Cno), UPDATE (Grade)
-> ON qyb.SC
-> TO 'tom'@'localhost';
Query OK, 0 rows affected (0.060 sec)
```

图 7-15 授予 tom 查询 Sno、Cno 的权限，以及修改 G 结果

如图 7-15 所示，该步骤展示了“授予 tom 查询 Sno、Cno 的权限，以及修改 G 结果”的执行结果。

测试查询：

```
mysql> SELECT Sno, Cno
-> FROM SC;
+-----+-----+
| Sno      | Cno      |
+-----+-----+
| 201705001 | CS101    |
| 201705002 | CS101    |
| 201705003 | CS101    |
| 201705001 | CS102    |
| 201705002 | CS102    |
| 201705003 | CS102    |
| 201705001 | CS201    |
| 201705002 | CS201    |
| 201705003 | CS201    |
| 201705001 | CS202    |
| 201705002 | CS202    |
| 201705001 | CS301    |
+-----+-----+
12 rows in set (0.016 sec)
```

图 7-16 测试查询

如图 7-16 所示，该步骤展示了“测试查询”的执行结果。

7.9 授予权限并允许继续授权

```
mysql> GRANT SELECT (Sno, Cno), UPDATE (Grade)
-> ON qyb.SC
-> TO 'tom'@'localhost'
-> WITH GRANT OPTION;
Query OK, 0 rows affected (0.333 sec)
```

图 7-17 7.9 授予权限并允许继续授权

如图 7-17 所示，该步骤展示了“7.9 授予权限并允许继续授权”的执行结果。

7.10 创建另一个用户并测试权限传递

```
mysql> CREATE USER 'jerry'@'localhost'
-> IDENTIFIED BY 'OpenGauss@2024';
Query OK, 0 rows affected (0.353 sec)
```

图 7-18 7.10 创建另一个用户并测试权限传递

如图 7-18 所示,该步骤展示了“7.10 创建另一个用户并测试权限传递”的执行结果。

7.11 回收列级权限

```
mysql> REVOKE SELECT (Sno, Cno), UPDATE (Grade)
-> ON qyb.SC
-> FROM 'tom'@'localhost';
Query OK, 0 rows affected (0.051 sec)
```

图 7-19 7.11 回收列级权限

如图 7-19 所示,该步骤展示了“7.11 回收列级权限”的执行结果。

tom 已经把权限传递给 **jerry**, 需要单独回收 **jerry** 的权限。

```
mysql> REVOKE SELECT (Sno)
-> ON qyb.SC
-> FROM 'jerry'@'localhost';
Query OK, 0 rows affected (0.332 sec)
```

图 7-20 tom 已经把权限传递给 jerry, 需要单独回收 je 结果

如图 7-20 所示,该步骤展示了“tom 已经把权限传递给 jerry, 需要单独回收 je 结果”的执行结果。

3. 实验分析与总结

分析:

本次实验主要练习了数据库安全性控制中的用户管理、角色管理、授权和回收授权操作。通过创建用户 **tom** 和角色 **jim**, 可以理解数据库用户和角色之间的关系: 用户是实际登录数据库并执行操作的主体, 而角色可以看作一组权限的集合, 可以将角色授予用户, 从而简化权限管理。在授权实验中, 通过 **GRANT** 语句可以为用户分配数据库、表或列级别的访问权限, 例如允许用户查询 **Teachers** 表, 或者只允许用户查询 **SC** 表中的 **Sno**、**Cno** 列并修改 **Grade** 列。通过这些操作可以体会到数据库安全控制的灵活性和精细化特点。

总结:

通过本次实验, 我掌握了创建用户、创建角色、授予权限和回收权限的基本方法, 并理解了数据库安全性控制的重要作用。数据库安全性控制可以限制不同用户对数据的访问范围, 防止越权查询、修改或删除数据, 从而保护数据库中的重要信息。通过本次

实验，我进一步理解了 GRANT 和 REVOKE 的使用方法，也加深了对数据库权限管理机制的认识。

问题及解决办法

问题：授权或回收权限后，用户权限变化不明显，容易混淆用户、角色和权限之间的关系。

解决办法：分别使用不同用户登录测试权限，并通过 GRANT 和 REVOKE 语句逐步验证授权效果。

心得体会：本实验加深了我对数据库安全性控制和权限管理机制的理解。

实验 8 事务与并发控制

1. 实验目的

理解事务的定义及其四个特点，掌握封锁机制的原理和方法，进一步理解并发、并发控制、封锁等相关概念。

2. 实验内容

- 1) 编写事务实例，分别使其成功提交和回滚，观察该事务对数据库的影响。
- 2) 利用数据库管理系统提供的封锁机制解决并发操作带来的数据不一致情况。
- 3) 编写事务实例，模拟两个事务产生死锁的情景。

2.1 创建实验数据库和表

```
mysql> CREATE DATABASE IF NOT EXISTS bank
-> DEFAULT CHARACTER SET utf8mb4
-> DEFAULT COLLATE utf8mb4_unicode_ci;
Query OK, 1 row affected (0.242 sec)

mysql>
mysql> USE bank;
Database changed
mysql>
```

图 8-1 2.1 创建实验数据库和表

如图 8-1 所示，该步骤展示了“2.1 创建实验数据库和表”的执行结果。

2.2 创建账户表 AccountInfo

```
mysql>
mysql> CREATE TABLE AccountInfo (
->     AccountNumber INT PRIMARY KEY,
->     BranchName VARCHAR(50),
->     Balance DECIMAL(10, 2)
-> ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
Query OK, 0 rows affected (0.694 sec)
```

图 8-2 2.2 创建账户表 AccountInfo

如图 8-2 所示，该步骤展示了“2.2 创建账户表 AccountInfo”的执行结果。

插入测试数据：

```
mysql> INSERT INTO AccountInfo(AccountNumber, BranchName, Balance)
-> VALUES
-> (1001, '光大河南总部', 15000.00),
-> (1002, '光大河南总部', 30000.00),
-> (1003, '光大洛阳分行', 25000.00),
-> (1004, '光大洛阳分行', 50000.00);
Query OK, 4 rows affected (0.172 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

图 8-3 插入测试数据

如图 8-3 所示，该步骤展示了“插入测试数据”的执行结果。

2.3 事务一：成功提交 COMMIT

模拟账户转账：从 1004 账户转出 20000 元，转入 1002 账户。

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.286 sec)

mysql>
mysql> UPDATE AccountInfo
-> SET Balance = Balance - 20000.00
-> WHERE AccountNumber = 1004;
Query OK, 1 row affected (0.069 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> UPDATE AccountInfo
-> SET Balance = Balance + 20000.00
-> WHERE AccountNumber = 1002;
Query OK, 1 row affected (0.010 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.329 sec)
```

图 8-4 模拟账户转账：从 1004 账户转出 20000 元，转结果

如图 8-4 所示，该步骤展示了“模拟账户转账：从 1004 账户转出 20000 元，转结果”的执行结果。

查看提交后的结果：

```
mysql> SELECT *
-> FROM AccountInfo
-> ORDER BY AccountNumber;
```

AccountNumber	BranchName	Balance
1001	光大河南总部	15000.00
1002	光大河南总部	50000.00
1003	光大洛阳分行	25000.00
1004	光大洛阳分行	30000.00

```
4 rows in set (0.014 sec)
```

图 8-5 查看提交后的结果

如图 8-5 所示，该步骤展示了“查看提交后的结果”的执行结果。

2.4 事务二：回滚 ROLLBACK

先执行一条修改语句，然后回滚。

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.007 sec)

mysql>
mysql> UPDATE AccountInfo
-> SET Balance = Balance - 10000.00
-> WHERE AccountNumber = 1004;
Query OK, 1 row affected (0.015 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> SELECT *
-> FROM AccountInfo
-> WHERE AccountNumber = 1004;
+-----+-----+-----+
| AccountNumber | BranchName | Balance |
+-----+-----+-----+
| 1004 | 光大洛阳分行 | 20000.00 |
+-----+-----+-----+
1 row in set (0.015 sec)

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.310 sec)
```

图 8-6 先执行一条修改语句，然后回滚

如图 8-6 所示，该步骤展示了“先执行一条修改语句，然后回滚”的执行结果。

回滚后再次查询：

```
mysql> SELECT *
-> FROM AccountInfo
-> WHERE AccountNumber = 1004;
+-----+-----+-----+
| AccountNumber | BranchName | Balance |
+-----+-----+-----+
| 1004 | 光大洛阳分行 | 30000.00 |
+-----+-----+-----+
1 row in set (0.016 sec)
```

图 8-7 回滚后再次查询

如图 8-7 所示，该步骤展示了“回滚后再次查询”的执行结果。

执行 **ROLLBACK** 后，事务中已经执行但尚未提交的修改被撤销，账户余额恢复到事务开始前的状态。

2.5 并发控制准备：创建 Customer 表

```
mysql>
mysql> CREATE TABLE Customer (
-> CustomerName VARCHAR(20) PRIMARY KEY,
-> CustomerStreet VARCHAR(50),
-> CustomerCity VARCHAR(20)
-> ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
Query OK, 0 rows affected (0.369 sec)
```

图 8-8 2.5 并发控制准备：创建 Customer 表

如图 8-8 所示，该步骤展示了“2.5 并发控制准备：创建 Customer 表”的执行结果。

插入数据：

```
mysql> SELECT *
-> FROM Customer;
+-----+-----+-----+
| CustomerName | CustomerStreet | CustomerCity |
+-----+-----+-----+
| 张伟         | 建设路123号   | 郑州         |
| 李娜         | 中原街456号   | 郑州         |
| 王强         | 牡丹大道789号 | 洛阳         |
| 陈英         | 唐街321号     | 洛阳         |
+-----+-----+-----+
4 rows in set (0.008 sec)
```

图 8-9 插入数据

如图 8-9 所示，该步骤展示了“插入数据”的执行结果。

2.6 并发实验一：READ COMMITTED 下的不可重复读

窗口 A

```
mysql> SELECT *
-> FROM Customer
-> WHERE CustomerName = '陈英';
+-----+-----+-----+
| CustomerName | CustomerStreet | CustomerCity |
+-----+-----+-----+
| 陈英         | 唐街321号     | 洛阳         |
+-----+-----+-----+
1 row in set (0.013 sec)

mysql>
```

图 8-10 窗口 A

如图 8-10 所示，该步骤展示了“窗口 A”的执行结果。

窗口 B

```
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.030 sec)

mysql>
mysql> UPDATE Customer
-> SET CustomerStreet = '唐街666号'
-> WHERE CustomerName = '陈英';
Query OK, 0 rows affected (0.007 sec)
Rows matched: 1 Changed: 0 Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.013 sec)

mysql>
```

图 8-11 窗口 B

如图 8-11 所示，该步骤展示了“窗口 B”的执行结果。

此时窗口 A 再次查询，会看到：

```
mysql> SELECT *
      -> FROM Customer
      -> WHERE CustomerName = '陈英';
+-----+-----+-----+
| CustomerName | CustomerStreet | CustomerCity |
+-----+-----+-----+
| 陈英         | 唐街666号      | 洛阳         |
+-----+-----+-----+
1 row in set (0.013 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.009 sec)
```

图 8-12 此时窗口 A 再次查询，会看到

如图 8-12 所示，该步骤展示了“此时窗口 A 再次查询，会看到”的执行结果。

在 **READ COMMITTED** 隔离级别下，同一个事务中两次读取同一条记录，可能因为其他事务提交了修改而读到不同结果，这就是不可重复读。

2.7 并发实验二：REPEATABLE READ 避免不可重复读

窗口 A

```
mysql>
mysql> SELECT *
      -> FROM Customer
      -> WHERE CustomerName = '陈英';
+-----+-----+-----+
| CustomerName | CustomerStreet | CustomerCity |
+-----+-----+-----+
| 陈英         | 唐街321号      | 洛阳         |
+-----+-----+-----+
1 row in set (0.012 sec)
```

图 8-13 窗口 A

如图 8-13 所示，该步骤展示了“窗口 A”的执行结果。

窗口 B

```
mysql>
mysql> UPDATE Customer
      -> SET CustomerStreet = '唐街666号'
      -> WHERE CustomerName = '陈英';
Query OK, 1 row affected (0.013 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.029 sec)

mysql>
```

图 8-14 窗口 B

如图 8-14 所示，该步骤展示了“窗口 B”的执行结果。

回到窗口 A

```
mysql> SELECT *
      -> FROM Customer
      -> WHERE CustomerName = '陈英';
+-----+-----+-----+
| CustomerName | CustomerStreet | CustomerCity |
+-----+-----+-----+
| 陈英         | 唐街321号     | 洛阳         |
+-----+-----+-----+
1 row in set (0.007 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.005 sec)
```

图 8-15 回到窗口 A

如图 8-15 所示，该步骤展示了“回到窗口 A”的执行结果。

在 MySQL 的 **REPEATABLE READ** 隔离级别下，窗口 A 在同一个事务中再次查询时，仍然看到事务开始时的数据，说明 **REPEATABLE READ** 可以避免不可重复读。

2.8 封锁机制实验：SELECT ... FOR UPDATE

窗口 A

```
mysql> USE bank;
Database changed
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.004 sec)

mysql>
mysql> UPDATE AccountInfo
      -> SET Balance = Balance + 10000.00
      -> WHERE AccountNumber = 1004;
```

图 8-16 窗口 A

如图 8-16 所示，该步骤展示了“窗口 A”的执行结果。

此时，1004 账户这一行被窗口 A 锁住。

窗口 B

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.005 sec)

mysql>
mysql> UPDATE AccountInfo
      -> SET Balance = Balance + 10000.00
      -> WHERE AccountNumber = 1004;
```

图 8-17 窗口 B

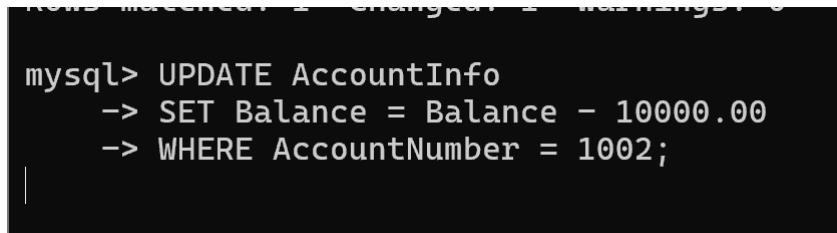
如图 8-17 所示，该步骤展示了“窗口 B”的执行结果。

这时窗口 B 会卡住等待，因为窗口 A 还没有提交或回滚。

2.9 死锁实验

窗口 A 想修改 1002，但 1002 被窗口 B 锁住，所以窗口 A 等待。

回到窗口 B

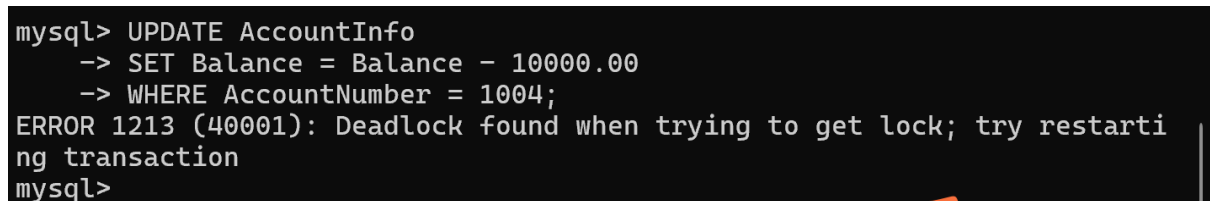


```
mysql> UPDATE AccountInfo  
-> SET Balance = Balance - 10000.00  
-> WHERE AccountNumber = 1002;
```

图 8-18 回到窗口 B

如图 8-18 所示，该步骤展示了“回到窗口 B”的执行结果。

窗口 B 想修改 1004，但 1004 被窗口 A 锁住，于是形成死锁。



```
mysql> UPDATE AccountInfo  
-> SET Balance = Balance - 10000.00  
-> WHERE AccountNumber = 1004;  
ERROR 1213 (40001): Deadlock found when trying to get lock; try restarting transaction  
mysql>
```

图 8-19 窗口 B 想修改 1004，但 1004 被窗口 A 锁结果

如图 8-19 所示，该步骤展示了“窗口 B 想修改 1004，但 1004 被窗口 A 锁结果”的执行结果。

3. 实验分析与总结

分析:本次实验主要围绕事务提交、事务回滚、事务隔离级别、封锁机制和死锁现象展开。通过 `START TRANSACTION`、`COMMIT` 和 `ROLLBACK` 操作，可以观察到事务对数据库数据的影响：当事务成功提交后，事务中的修改会永久保存到数据库中；当事务回滚后，事务中尚未提交的修改会被撤销，数据恢复到事务开始前的状态。这体现了事务的原子性，即一个事务中的多个操作要么全部成功，要么全部失败。

总结:通过本次实验，我进一步理解了事务的四个基本特性，即原子性、一致性、隔离性和持久性。事务机制可以保证数据库在执行一组相关操作时保持数据完整，特别是在转账等场景中，必须保证转出和转入两个操作同时成功或同时失败，否则会造成数据不一致。

问题及解决办法

问题：并发操作同一数据时，可能出现等待、阻塞或死锁现象。

解决办法：按照固定顺序访问数据资源，及时提交或回滚事务，减少事务长时间占用锁。

心得体会：本实验使我理解了事务隔离级别、锁机制和并发控制对数据库一致性的保障作用。

实验 9 数据库的备份与恢复

1. 实验目的

理解各种数据库故障以及它们的恢复方法，掌握数据库备份原理和方法，以及如何使用备份数据进行数据库故障的恢复。

2. 实验内容

- 1) 针对具体故障制定备份方案，并利用这些备份方案恢复数据库。
- 2) 使用 OpenGauss 提供的各种备份方法进行数据库备份。
- 3) 使用具体的数据库管理系统提供的恢复机制，利用副本进行数据库的恢复。

2.1 创建实验数据库

```
exp9_backup=# CREATE TABLE backup_test (  
exp9_backup(#      id SERIAL PRIMARY KEY,  
exp9_backup(#      sname VARCHAR(50),  
exp9_backup(#      major VARCHAR(50),  
exp9_backup(#      create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
exp9_backup(# );  
NOTICE: CREATE TABLE will create implicit sequence "backup_test_id_seq"  
for serial column "backup_test.id"  
NOTICE: CREATE TABLE / PRIMARY KEY will create implicit index "backup_t  
est_pkey" for table "backup_test"  
CREATE TABLE
```

图 9-1 2.1 创建实验数据库

如图 9-1 所示，该步骤展示了“2.1 创建实验数据库”的执行结果。

2.2 创建测试表并插入数据

```
exp9_backup=# INSERT INTO backup_test(sname, major)  
exp9_backup-# VALUES  
exp9_backup-# ('张三', '计算机科学与技术'),  
exp9_backup-# ('李四', '口腔医学'),  
exp9_backup-# ('王五', '人工智能');  
INSERT 0 3  
exp9_backup=# SELECT * FROM backup_test;  
 id | sname |      major      |      create_time  
----+-----+-----+-----  
  1 | 张三  | 计算机科学与技术 | 2026-06-03 09:16:43.067811  
  2 | 李四  | 口腔医学         | 2026-06-03 09:16:43.067811  
  3 | 王五  | 人工智能         | 2026-06-03 09:16:43.067811  
(3 rows)
```

图 9-2 2.2 创建测试表并插入数据

如图 9-2 所示，该步骤展示了“2.2 创建测试表并插入数据”的执行结果。

2.3 使用 gs_dump 进行逻辑备份

创建备份目录

```
[omm@db1 ~]$ mkdir -p /home/omm/backup_exp9
[omm@db1 ~]$ ls -l /home/omm
total 4
drwx-----. 2 omm dbgrp 4096 Jun  3 09:17 backup_exp9
[omm@db1 ~]$
```

图 9-3 创建备份目录

如图 9-3 所示，该步骤展示了“创建备份目录”的执行结果。

备份数据库，备份完成后查看文件

```
[omm@db1 ~]$ gs_dump -f /home/omm/backup_exp9/exp9_backup.sql exp9_backup -p 26000
gs_dump[port='26000'][exp9_backup][2026-06-03 09:19:30]: The total objects number is 436.
gs_dump[port='26000'][exp9_backup][2026-06-03 09:19:30]: [100.00%] 436 objects have been dumped.
gs_dump[port='26000'][exp9_backup][2026-06-03 09:19:30]: dump database exp9_backup successfully
gs_dump[port='26000'][exp9_backup][2026-06-03 09:19:30]: total time: 4934 ms
[omm@db1 ~]$ ls -lh /home/omm/backup_exp9
total 4.0K
-rw-----. 1 omm dbgrp 2.2K Jun  3 09:19 exp9_backup.sql
```

图 9-4 备份数据库，备份完成后查看文件

如图 9-4 所示，该步骤展示了“备份数据库，备份完成后查看文件”的执行结果。

2.4 模拟数据库故障

模拟误删表，并验证表已经被删除

```
exp9_backup=# DROP TABLE backup_test;
DROP TABLE
exp9_backup=# \dt
No relations found.
exp9_backup=#
```

图 9-5 模拟误删表，并验证表已经被删除

如图 9-5 所示，该步骤展示了“模拟误删表，并验证表已经被删除”的执行结果。

2.5 使用备份文件恢复数据库

2.5.1 方法一：直接恢复到原数据库

因为原数据库 exp9_backup 还存在，只是表被删除，所以可以直接把备份 SQL 导入原数据库：

```
[omm@db1 ~]$ gsql -d exp9_backup -p 26000 -f /home/omm/backup_exp9/exp9_
backup.sql
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
ALTER TABLE
setval
-----
      3
(1 row)

ALTER TABLE
REVOKE
REVOKE
GRANT
GRANT
total time: 217 ms
```

图 9-6 因为原数据库 exp9_backup 还存在，只是表被删结果

如图 9-6 所示，该步骤展示了“因为原数据库 exp9_backup 还存在，只是表被删结果”的执行结果。

验证恢复结果

```
exp9_backup=# SELECT * FROM backup_test;
 id | sname | major | create_time
-----+-----+-----+-----
  1 | 张三 | 计算机科学与技术 | 2026-06-03 09:16:43.067811
  2 | 李四 | 口腔医学 | 2026-06-03 09:16:43.067811
  3 | 王五 | 人工智能 | 2026-06-03 09:16:43.067811
(3 rows)
```

图 9-7 验证恢复结果

如图 9-7 所示，该步骤展示了“验证恢复结果”的执行结果。

2.5.2 方法二：完整恢复方案：删除数据库后恢复

删除实验数据库

```
openGauss=# DROP DATABASE exp9_backup;
DROP DATABASE
```

图 9-8 删除实验数据库

如图 9-8 所示，该步骤展示了“删除实验数据库”的执行结果。

重新创建空数据库

```
openGauss=# CREATE DATABASE exp9_backup;
CREATE DATABASE
```


图 9-9 重新创建空数据库

如图 9-9 所示，该步骤展示了“重新创建空数据库”的执行结果。

导入备份文件恢复

```
[omm@db1 ~]$ gsql -d exp9_backup -p 26000 -f /home/omm/backup_exp9/exp9_
backup.sql
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
ALTER TABLE
setval
-----
      3
(1 row)

ALTER TABLE
REVOKE
REVOKE
GRANT
GRANT
total time: 408 ms
```

图 9-10 导入备份文件恢复

如图 9-10 所示，该步骤展示了“导入备份文件恢复”的执行结果。

验证

```
exp9_backup=# \dt
M backup_test;
List of relations
Schema | Name | Type | Owner | Storage
-----+-----+-----+-----+-----
public | backup_test | table | omm | {orientation=row,compression=no}
(1 row)

exp9_backup=# SELECT * FROM backup_test;
 id | sname | major | create_time
-----+-----+-----+-----
  1 | 张三 | 计算机科学与技术 | 2026-06-03 09:16:43.067811
  2 | 李四 | 口腔医学 | 2026-06-03 09:16:43.067811
  3 | 王五 | 人工智能 | 2026-06-03 09:16:43.067811
(3 rows)
```

图 9-11 验证

如图 9-11 所示，该步骤展示了“验证”的执行结果。

表和数据恢复成功，说明整个恢复流程完成。

3. 实验分析与总结

分析：通过本次实验可以看出，数据库备份是保证数据库安全性和可靠性的重要措

施。当数据库发生误删除、误修改或系统故障时，如果提前进行了有效备份，就可以利用备份文件将数据库恢复到正常状态。openGauss 中的 `gs_dump` 工具属于逻辑备份工具，它可以将数据库对象结构和数据导出为 SQL 文件，恢复时再通过 `gsql` 执行该文件，从而重建数据库表和数据内容。逻辑备份的优点是操作简单、文件可读性强、便于迁移和保存，适合课程实验和小规模数据库系统的数据保护。

总结：通过本次实验，我进一步理解了数据库故障恢复的基本思想：先建立可用备份，再在发生故障后根据备份文件恢复数据库对象和数据。对于实际应用而言，应根据数据重要程度制定周期性备份方案，例如每日逻辑备份、定期全量备份以及必要的日志备份。同时，还应定期验证备份文件是否可用，避免出现“有备份但无法恢复”的问题。本次实验提高了我对数据库安全维护和故障处理的认识，也为后续学习数据库恢复机制、日志恢复和事务管理打下了基础。

问题及解决办法

问题：恢复数据库时，可能因备份路径、数据库名或导入命令错误导致恢复失败。

解决办法：确认备份文件位置正确，重新创建目标数据库，并使用导入命令恢复后查询验证。

心得体会：本实验让我认识到数据库备份与恢复是保障数据安全的重要手段，应在实际应用中制定合理备份策略。